1. Scenario

You have just been hired as **data manager** at a hospital. Clinical data arrives from the hospital information system as **FHIR R4 JSON bundles** (one file per patient). Several groups need access to these data, but **not to the same data, and not with the same rights**:

Who	What they need	What they must *not* be able to do
Clinician	Read full records of patients; add clinical comments	Delete or rewrite clinical facts
Patient	Read *their own* record	See any other patient; modify anything
Data manager (you)	Register new patients; run the ingestion pipeline; monitor data quality	Overwrite clinical observations silently
Data analyst *(Phase 2)*	Extract anonymized data to build dashboards and detect anomalies	Re-identify patients

Your job over the project is to build the **data pipeline** and the **REST API** that serve these users.

Scope note about security. Authentication, encryption, and secure deployment are essential in a real hospital (GDPR, French *Hébergement de Données de Santé* certification) but are **out of scope** here.

---

2. The data

- **Source:** *Synthea Dataset Jsons – EHR* available following this link : Synthea dataset.
- **Nature of the data: synthetic** patients generated by Synthea. No real person is behind these records, which is why we can use them freely. They nonetheless look realistic (names, addresses, birth dates), so de-identification in Phase 2 is a genuine exercise.
- **Format:** HL7 **FHIR R4** (Fast Healthcare Interoperability Resources, https://hl7.org/fhir/,)
- **Key difficulty:** FHIR is designed for **exchange**, not analysis. It is deeply nested (lists of dictionaries inside lists). Turning it into queryable tables is the first real data-management task of this project.

---

3. The project in the language of data pipelines

ETL vs ELT - you will have to justify your choice.

- **ETL:** flatten JSON in Python *before* loading → the database only holds clean tables.
- **ELT:** load raw JSON as-is, then transform with SQL inside the database

Principle to keep in mind: in a hospital, the clinical record is a **legal document**. The API never overwrites clinical facts. Users *add* information (comments, new patients); every addition is **traceable** (who, which role, when).

---

4. Phase 1: model design and API creation

4.1 Data model (minimum)

There is no imposed schema. Your relational model must satisfy these requirements:

- Entities. Cover at least the patients, their encounters (visits / stays), at least two types of clinical facts of your choice (e.g. diagnoses, measurements, medications, procedures), and the comments added through the API.
- Identity. Each table has a primary key. Use the FHIR id for ingested data; the server generates identifiers for records created through the API.
- Clinical facts are interpretable. For each fact, keep what (code, coding system, human-readable label), when, and, if relevant, the value with its unit.
- Traceability of additions. Comments are append-only and record who wrote them, with which role, when, and what they refer to.

4.2 Endpoints and permissions (minimum)

Method & path	Purpose	clinician	patient	data_manager
`GET /patients`	List patients	yes	no	yes
`GET /patients/{id}`	Patient summary	yes	own only	yes
`GET /patients/{id}/encounters`	Encounters of a patient	yes	own only	yes
`GET /patients/{id}/observations?code=…`	Observations	yes	own only	yes
`GET /patients/{id}/comments`	Comments on a record	yes	own only	yes
`POST /patients/{id}/comments`	**Add** a comment (append-only)	yes	no	no
`POST /patients`	Register a new patient	no	no	yes

Rules:

1. **Use error codes** from the HTTP standard, consistently across all endpoints.
2. **There is no endpoint that modifies or deletes clinical data.** The only way to “update” a record is to **add** a comment.

4.3 Requirements

- Automatic documentation via FastAPI `/docs` with meaningful descriptions.